

Projet synthèse

420-C61-VM
420-C61-IN

Charlotte DUPRAS
Eddy HUART

DOCUMENT DE CONCEPTION — L'ATTAQUE

Diagrammes et éléments de conception

Présenté à Pierre-Paul Monty

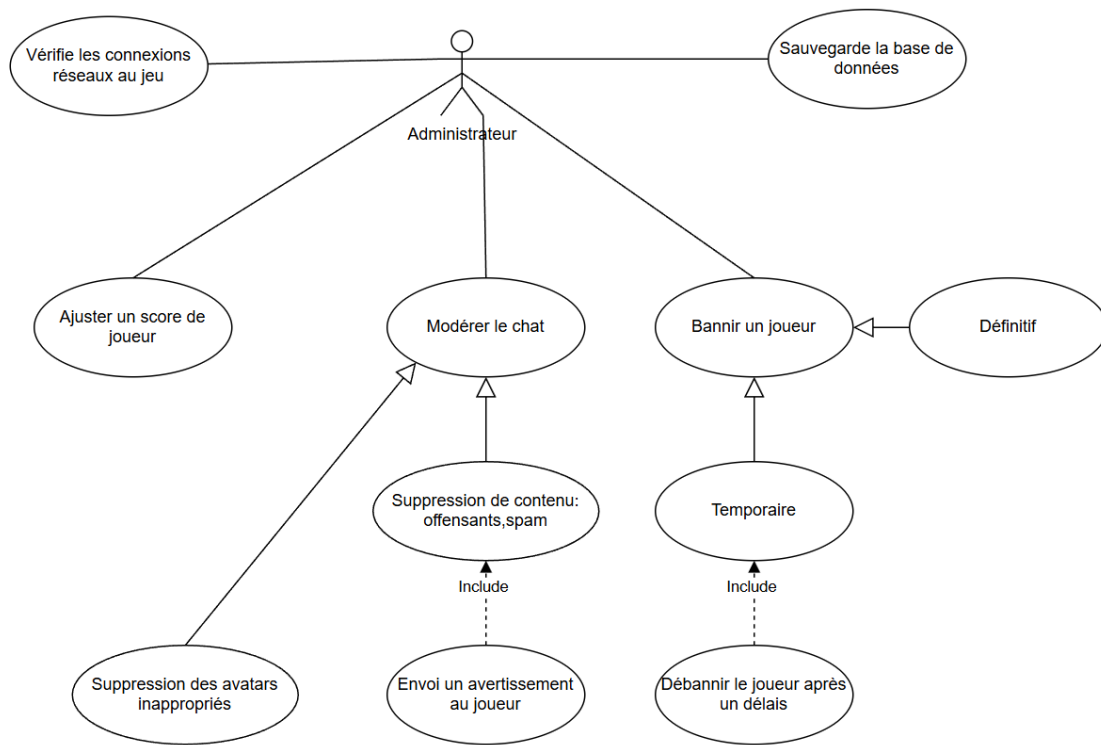
vendredi, 27 février 2026

Table des matières

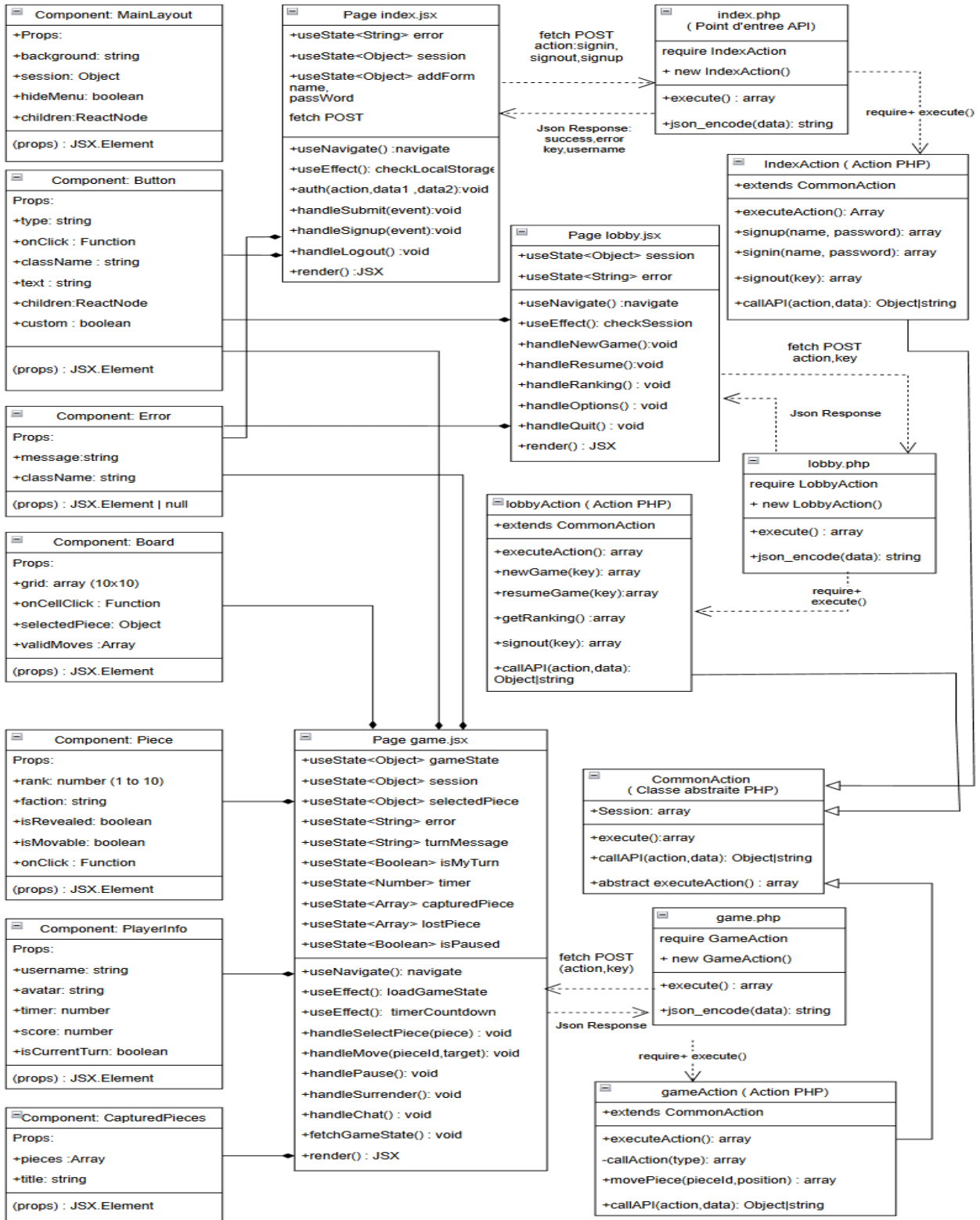
Diagrammes de cas d'usage	3
Diagrammes de classes	6
Modèle relationnel	8
Maquettes	9
Éléments de conception.....	14
Structures de données.....	14
Structure arborescente	14
Dictionnaire	14
Tableau	14
Patrons de conception.....	15
Factory Pattern	15
Observer.....	15
MVC	15
Intégration individuelle	16
Choix et implémentation de React.....	16
Choix et implémentation de l'algorithme MCTS.....	16

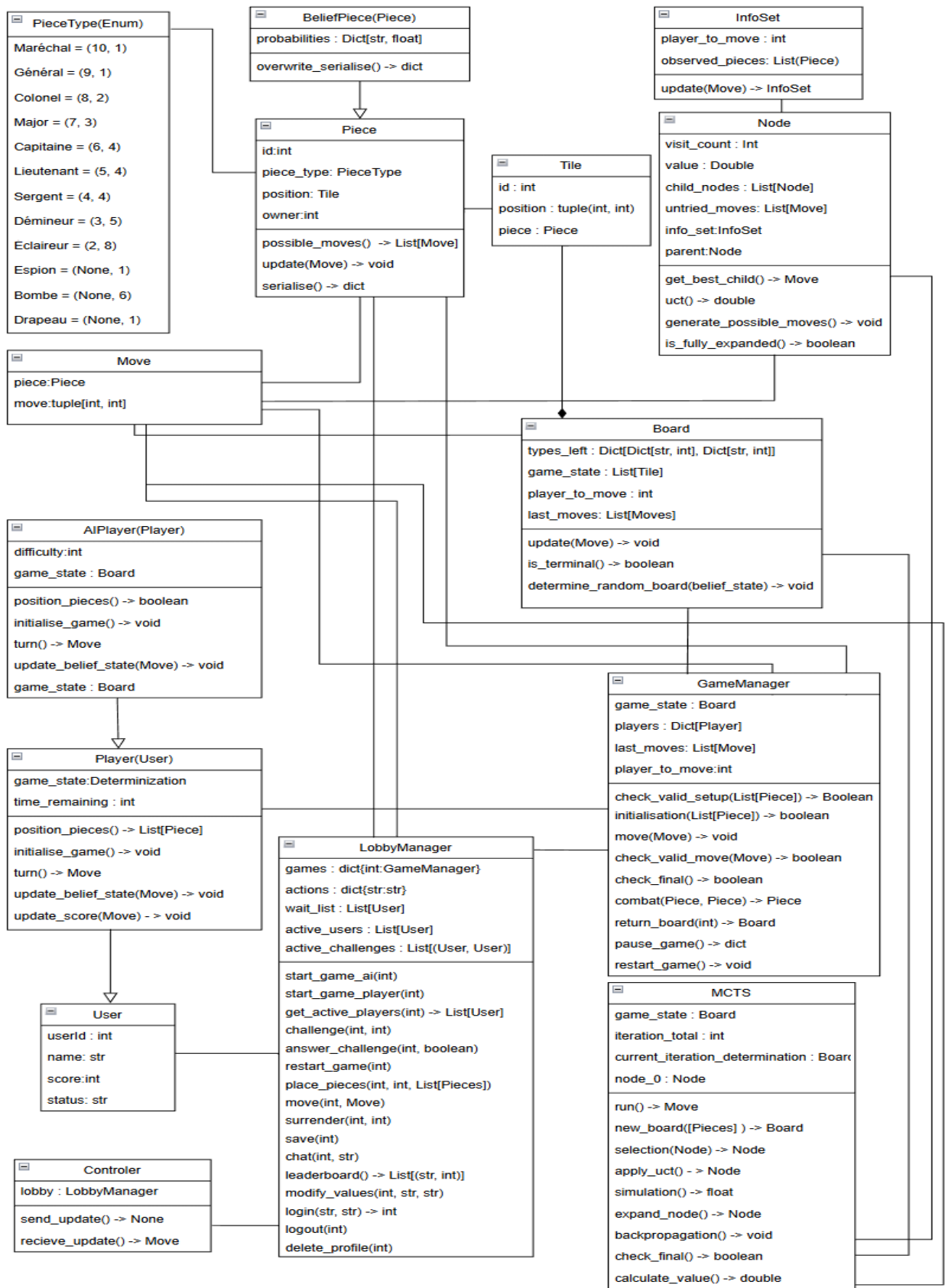
Diagrammes de cas d'usage



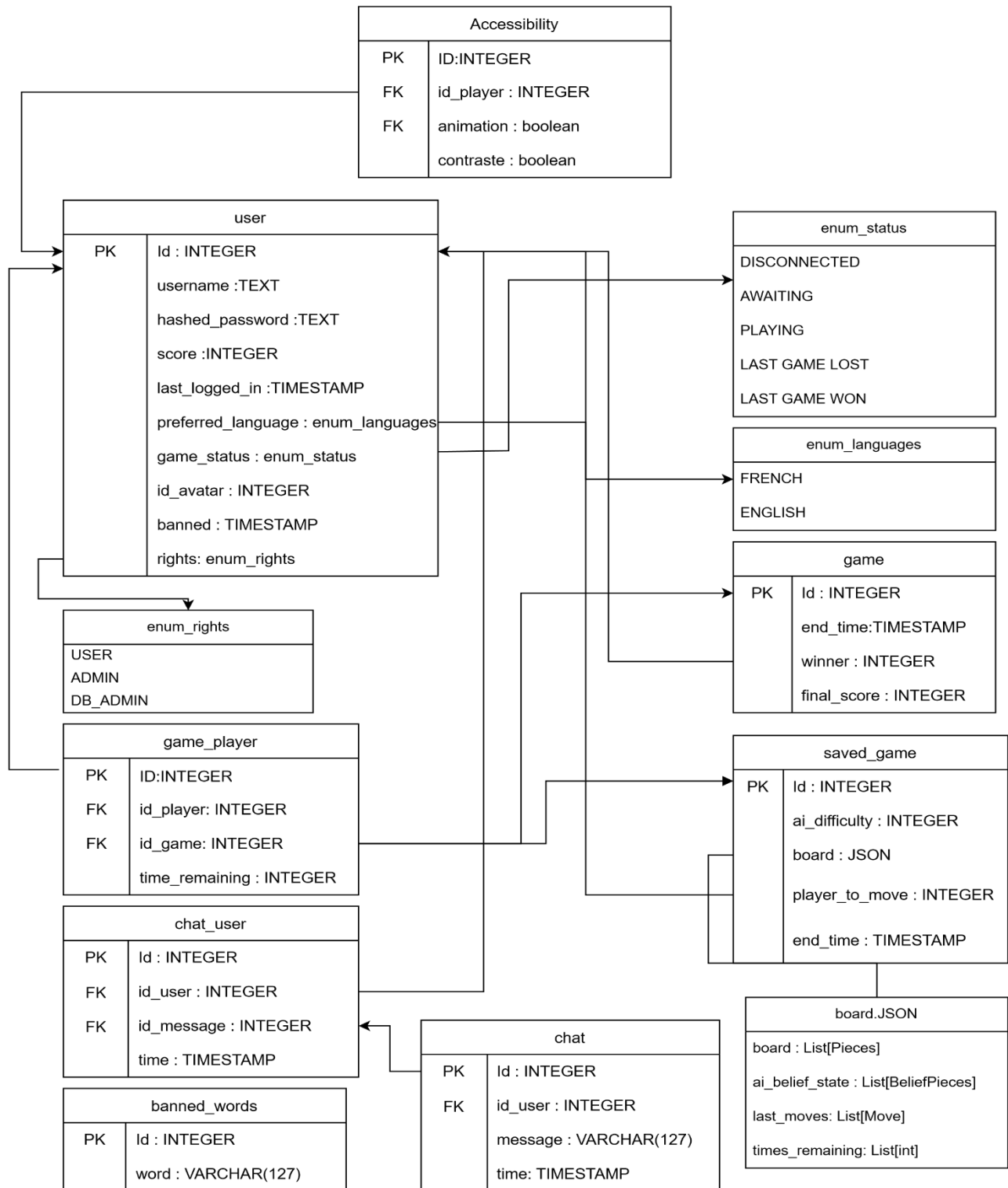


Diagrammes de classes





Modèle relationnel



Maquettes

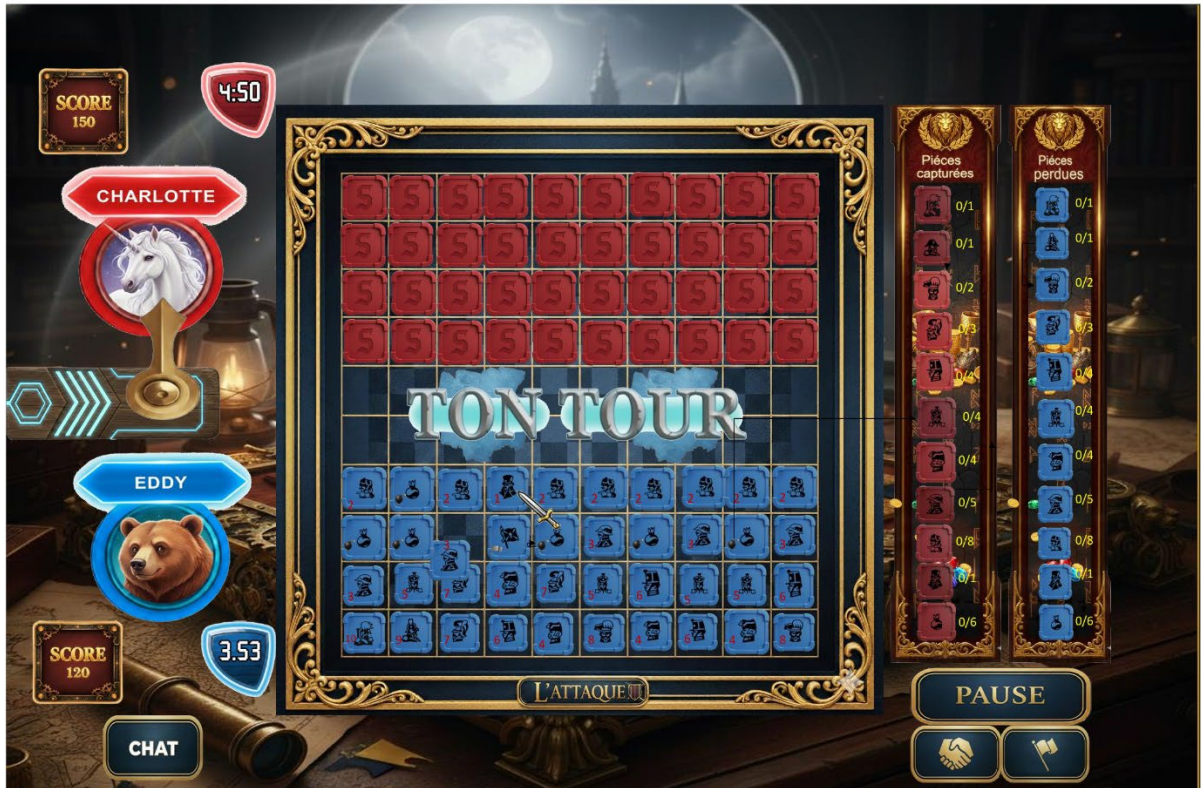
ECRAN CONNEXION



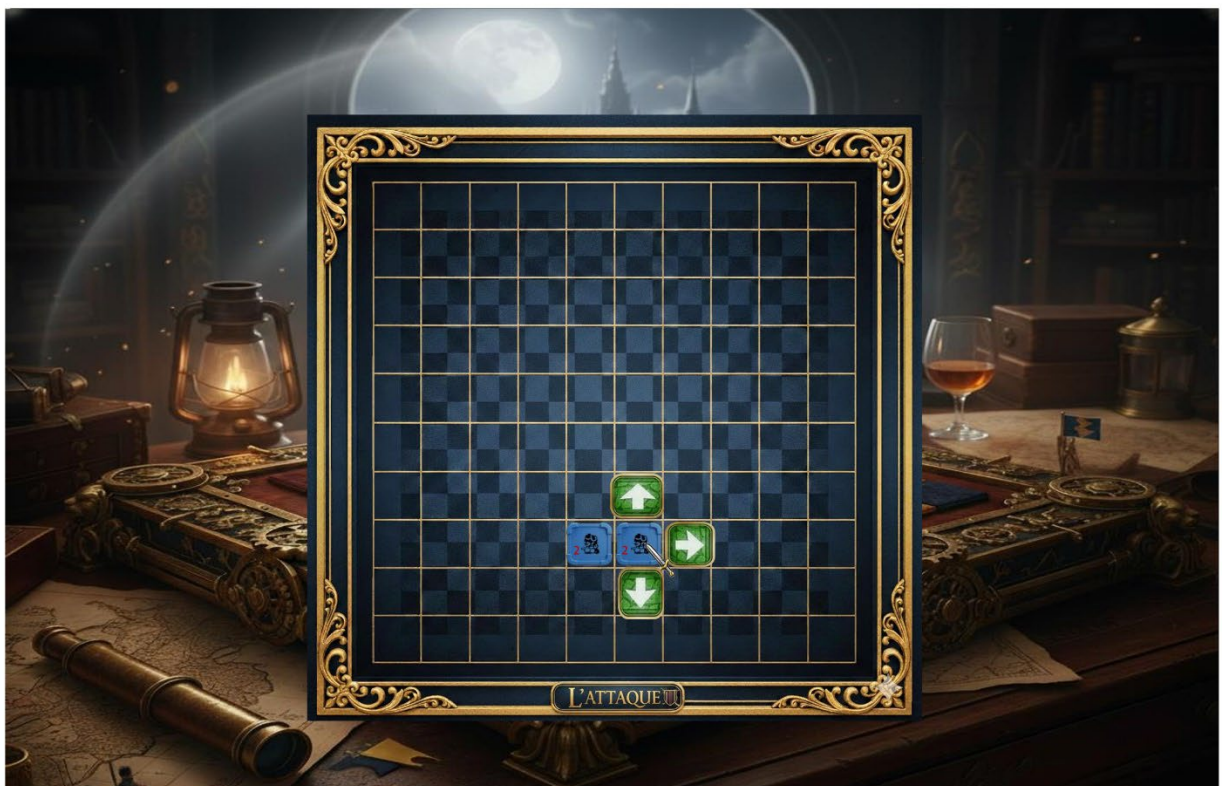
ECRAN PRINCIPAL



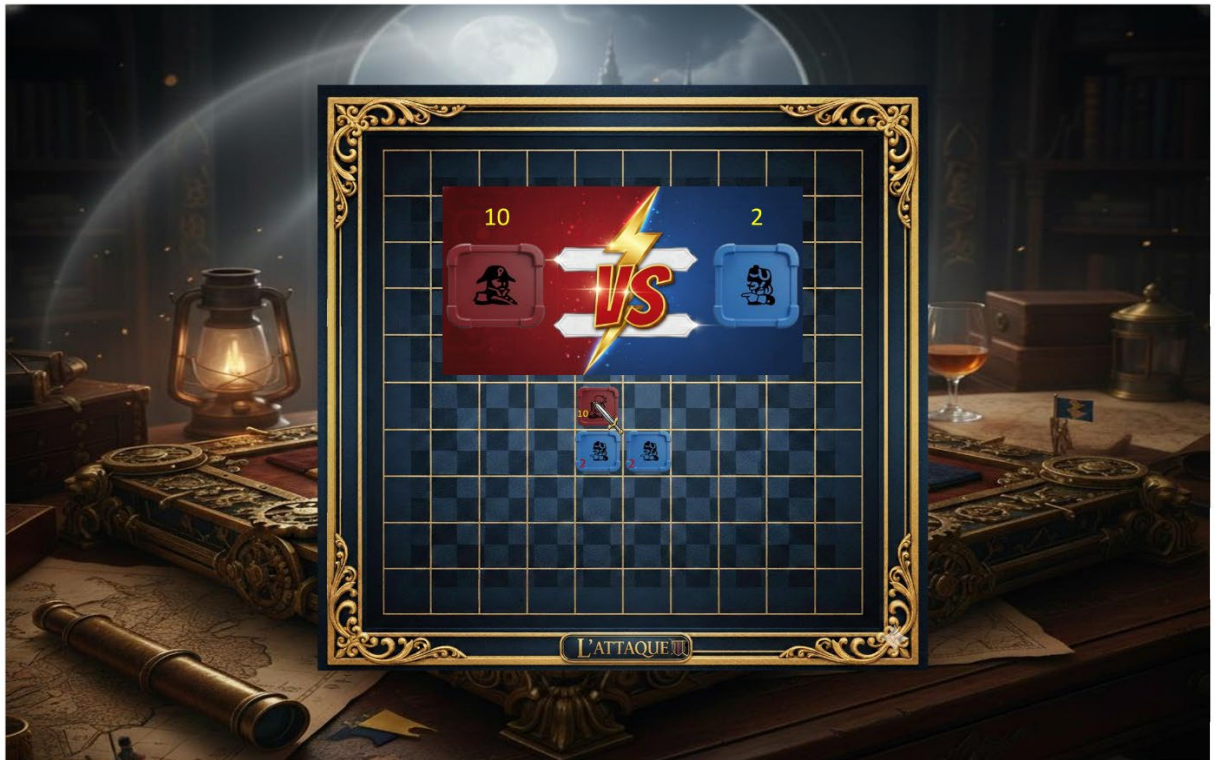
ECRAN JEU



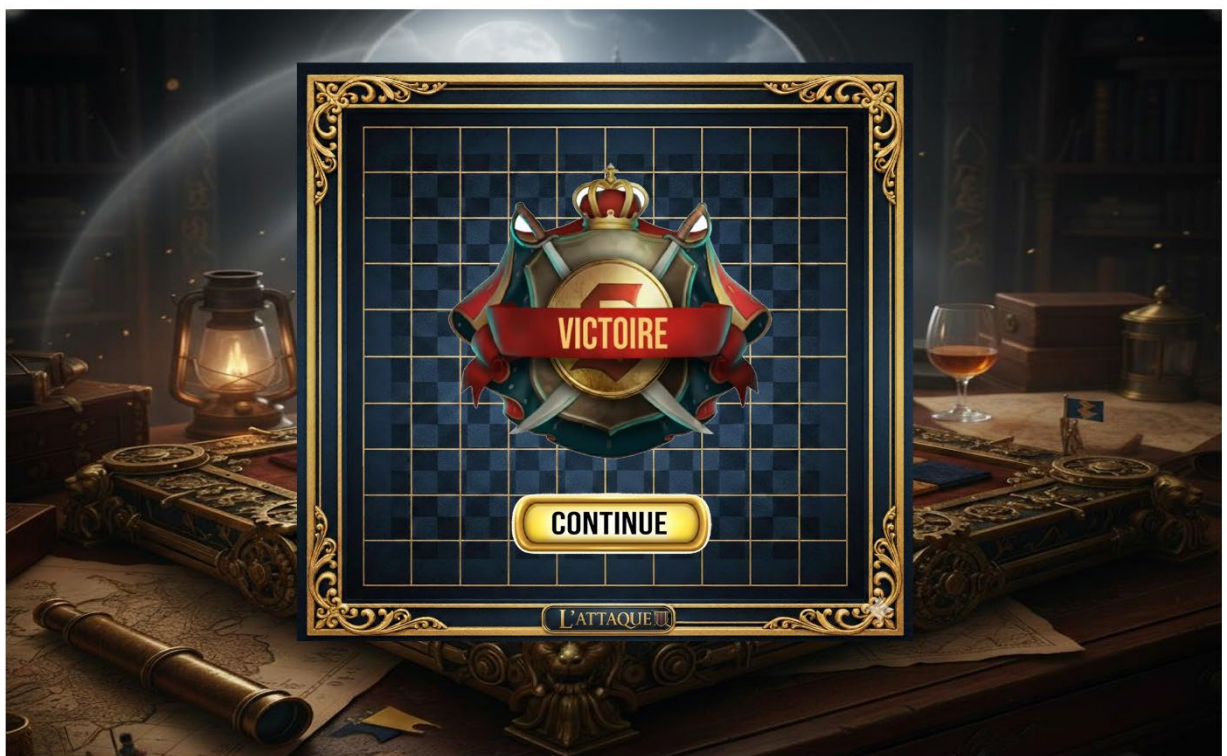
DEPLACEMENT DE PIECE



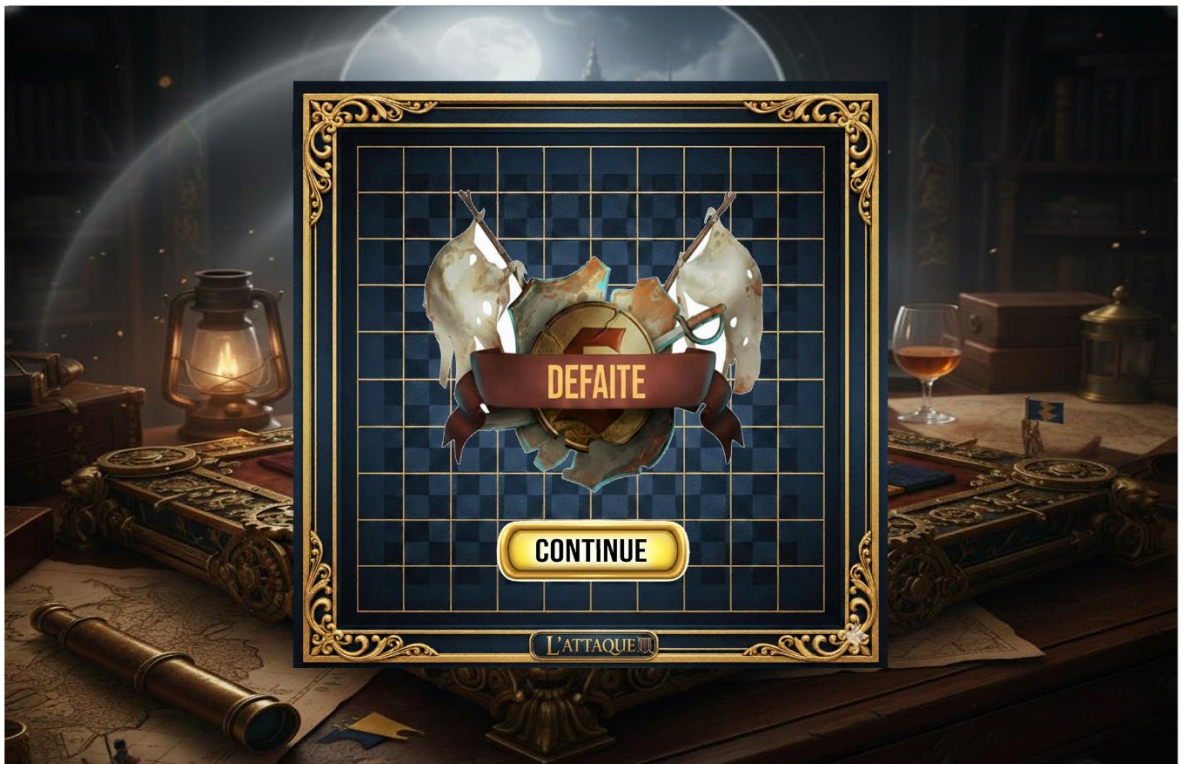
COMBAT



VICTOIRE



DEFAITE



OPTIONS



CLASSEMENT

CLASSEMENT

1	Nom du Joueur	2950
2	Nom du Joueur	1250
3	Nom du Joueur	950
4	Nom du Joueur	850
5	Nom du Joueur	600

Éléments de conception

Structures de données

Structure arborescente

Nous utiliserons une structure arborescente pour représenter l'arbre de recherche généré par l'algorithme MCTS. Chaque nœud contiendra une référence vers son parent et une liste de ses enfants, ce qui permet de maintenir les relations hiérarchiques entre les différents états du jeu. Cette organisation facilite la navigation dans l'arbre. Elle est particulièrement utile lors de la phase de rétropropagation à la fin d'une itération, puisqu'elle permet de remonter efficacement vers les nœuds précédents pour mettre à jour leurs statistiques.

Dictionnaire

Dans la classe *BeliefPiece*, la variable *probabilities* sera représentée par un dictionnaire. Les clés correspondront aux différents types de pièces (Espion, Maréchal, etc.) et les valeurs seront des nombres réels compris entre 0 et 1, représentant la probabilité que la pièce donnée corresponde au type défini par la clé. Ce dictionnaire est intéressant dans le cadre de notre projet, puisqu'il permet à l'IA de conserver un état de croyance (Belief State) persistant dans le temps. Les valeurs du dictionnaire seront régulièrement modifiées, puisque le mouvement d'une pièce ou l'identification de celle-ci viendra modifier la valeur de ses clés. Ce type de données est donc adéquat pour organiser l'État de Croyance de l'IA.

Tableau

Pendant la communication entre le serveur et le client, un tableau contenant l'ensemble des pièces du jeu sera transmis aux joueurs. Cette approche permet d'envoyer toutes les données en une seule structure plutôt que d'effectuer 80 envois séparés. Cela réduit la fréquence des transferts et améliore l'efficacité des communications.

Patrons de conception

Factory Pattern

Ce patron de conception permet d'ajouter facilement différents thèmes graphiques aux pièces sans devoir coder des classes distinctes pour chacune d'entre elles. Il réduit la complexité du code (moins de branchements conditionnels) en centralisant la logique commune dans une fonction usine. Il permet donc de séparer la logique du jeu de l'apparence des pièces, ce qui est particulièrement intéressant puisque les pièces partagent des comportements similaires, mais ont des styles graphiques différents.

Observer

L'utilisation de `useEffect` pour gérer le changement de tour correspond à un patron Observer. Après un mouvement validé de la part d'un joueur, le serveur notifie l'adversaire du changement de tour. La variable contenant le tour actif (sujet) est surveillée par `useEffect` (Observer). Si sa valeur change, le composant est rafraîchi automatiquement. Ce mécanisme permet une mise à jour réactive et non impérative de l'interface.

MVC

Nous utiliserons l'architecture Modèle-Vue-Contrôleur (MVC). Le modèle, codé en Python, contiendra les données et la logique du jeu. Il sera indépendant de l'interface utilisateur et gèrera le plateau et les règles, ainsi que l'intelligence artificielle et les attributs des joueurs. La vue, codée en React, recevra uniquement ce qui lui est nécessaire du contrôleur. Elle lui transmettra, en retour, les actions du joueur. Finalement, le contrôleur, situé dans le backend, fera le lien entre le modèle et la vue en assurant la transmission des données.

Intégration individuelle

Choix et implémentation de React

L'interface utilisateur de ce projet sera développée avec React, une bibliothèque JavaScript idéale pour créer des applications interactives. Dans un jeu de type Stratego, les informations évoluent constamment (déplacement d'un pion, écoulement du temps, capture d'une pièce, etc.). Sans mécanisme d'optimisation, plusieurs éléments pourraient être nus à jour inutilement, ce qui nuirait à la performance.

React utilise un DOM virtuel : il détecte précisément ce qui a changé et ne met à jour que l'élément concerné, comme le déplacement d'une seule pièce sur le plateau. Cela garantit que le jeu reste fluide, même lors d'actions rapides.

De plus, au lieu d'avoir un seul bloc de code massif, React permet de diviser l'interface en petites pièces indépendantes : les composants (le plateau, les pièces, le score, etc.). Ainsi, pour modifier l'apparence des pions, il suffit de changer un seul fichier sans affecter le reste du système. Cette approche rend le projet beaucoup plus facile à construire, à tester et à faire évoluer dans l'avenir.

Choix et implémentation de l'algorithme MCTS

Les joueurs auront l'option de jouer contre une intelligence artificielle en temps réel. L'implémentation de cette IA sera basée sur l'algorithme Monte Carlo Tree Search (MCTS), puisqu'il peut produire un résultat sans faire une évaluation exhaustive de l'espace de recherche. Cette approche est particulièrement adaptée à Stratego, où le nombre de positions possibles est extrêmement élevé.

Grâce à la mécanique d'exploration/exploitation, l'IA peut rapidement déterminer quelles branches (mouvements) de son arbre de recherche sont les plus intéressantes et se concentrer sur celles-ci. Il évite ainsi de perdre du temps sur des mouvements qui ne lui rapporteront rien et peut donc se concentrer sur l'évaluation des conséquences possibles de mouvements compétitifs.

Pour permettre à l'algorithme d'effectuer ses recherches dans un jeu à information imparfaite (l'identité des pièces ennemies n'est pas connue avant l'issue d'un combat), l'algorithme sera combiné à des Belief States. Ces états représenteront les hypothèses de l'IA concernant l'identité des pièces adverses. Ils seront mis à jour après chaque mouvement ou combat, en

fonction des informations observées. L'IA utilisera ces états pour affiner ses décisions stratégiques.